

Table des matières

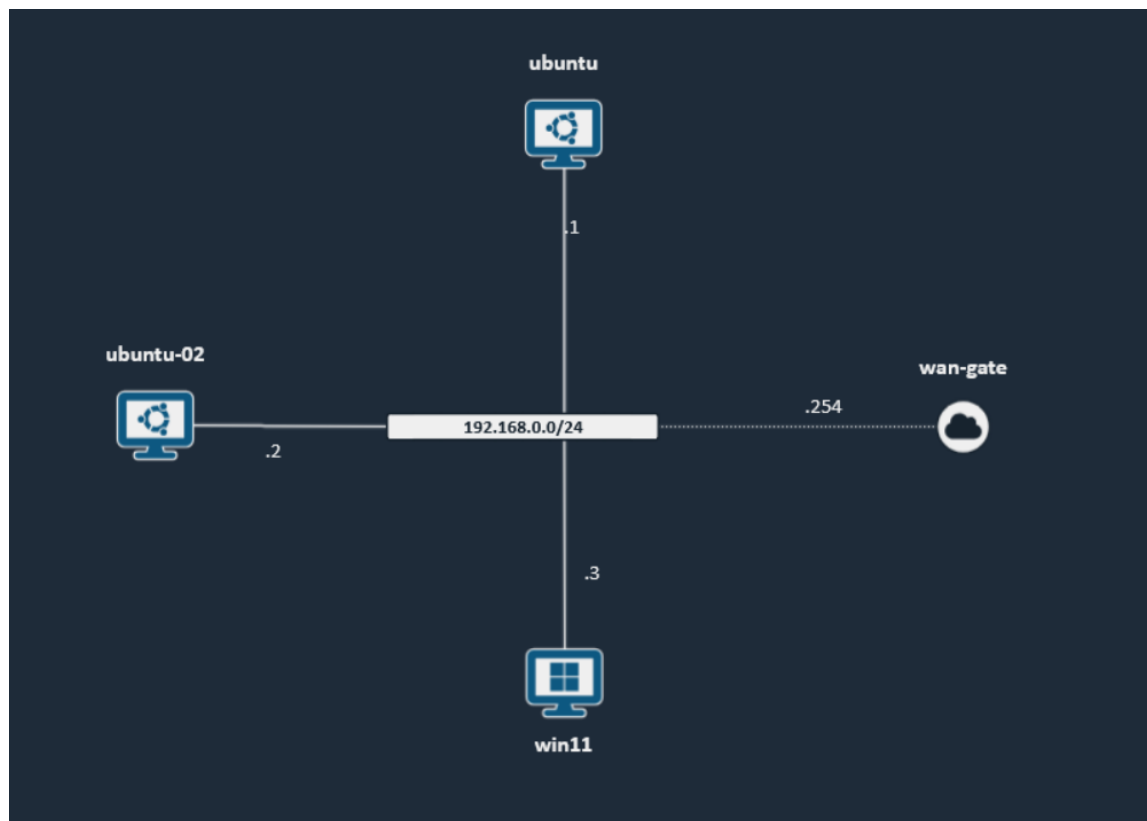
Introduction :	4
Rôle du serveur principal	4
Attaque par brute force	14
Installation de Hydra	14
Creation du dictionnaire	14
Lancement de l'attaque	15
Detection sur Wazuh	15
Surveillance de l'integrite des fichiers	16
Configuration sur l'agent Linux	16
Configuration sur l'agent Windows	17
Detection sur Wazuh	17
Integration de VirusTotal	18
Compte et cle API	18
Ajout de l'integration sur le manager	19
Dossier surveille en temps reel	19
Redemarrage des services	20
Test avec le fichier EICAR	20
Detection sur Wazuh	20
Mise en place de Suricata	21
Installation via le depot officiel OISF	21
Ajout des regles Emerging Threats	21
Configuration de suricata.yaml	21
Redemarrage et verification	22
Attaque reseau par deni de service	22
Remontee des alertes Suricata vers Wazuh	23
Attaque reseau par scan de ports	24
Deploiement de DVWA	26
Installation de l'environnement	26
Base de donnees	26
Recuperation et configuration de DVWA	27
Surveillance des logs Apache	28
Initialisation et acces a DVWA	28
Attaque web par XSS reflechi	31
Attaque web par injection SQL	33

Regles de detection personnalisees : detection d'une enumeration de repertoires.....	34
Test de la regle	35
Conclusion.....	37

Déploiement de Wazuh, Détection d'Attaques et Analyse de Menaces

Introduction :

Ce rapport est réalisé dans le cadre du cours SOC à Efrei, destiné aux étudiants de la filière Cloud and Computing. Il présente une simulation sur la plateforme CyberRange d'Airbus, incluant l'installation du SIEM Wazuh, la détection d'attaques brute force, la surveillance d'intégrité des fichiers (FIM), et l'intégration de VirusTotal pour l'analyse de fichiers suspects, la détection d'attaques réseau et web à l'aide des outils Suricata (IDS/IPS).



Rôle du serveur principal

Le serveur principal regroupe plusieurs composants clés : le Wazuh Manager, l'Indexer (basé sur OpenSearch) et Filebeat. Chaque agent installé sur les machines clientes remonte ses journaux d'événements vers ce serveur centralisé.

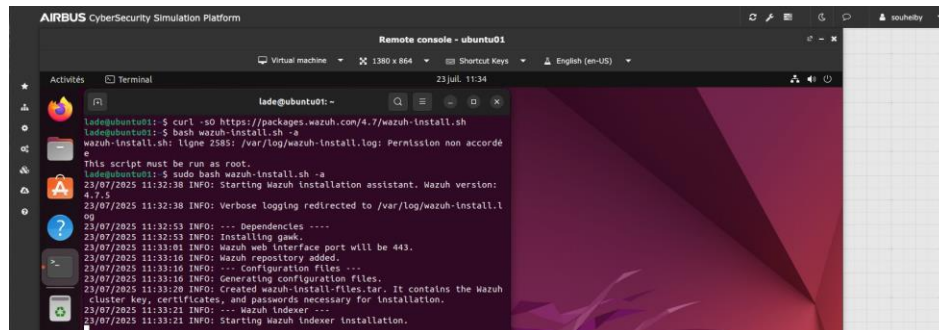
Ce dernier dispose également d'une interface graphique intuitive, permettant de visualiser en temps réel les événements de sécurité, l'état des machines surveillées et la configuration des agents.

Pour son installation, nous avons mis à jour les paquets :

```
sudo apt update && sudo apt upgrade -y
```

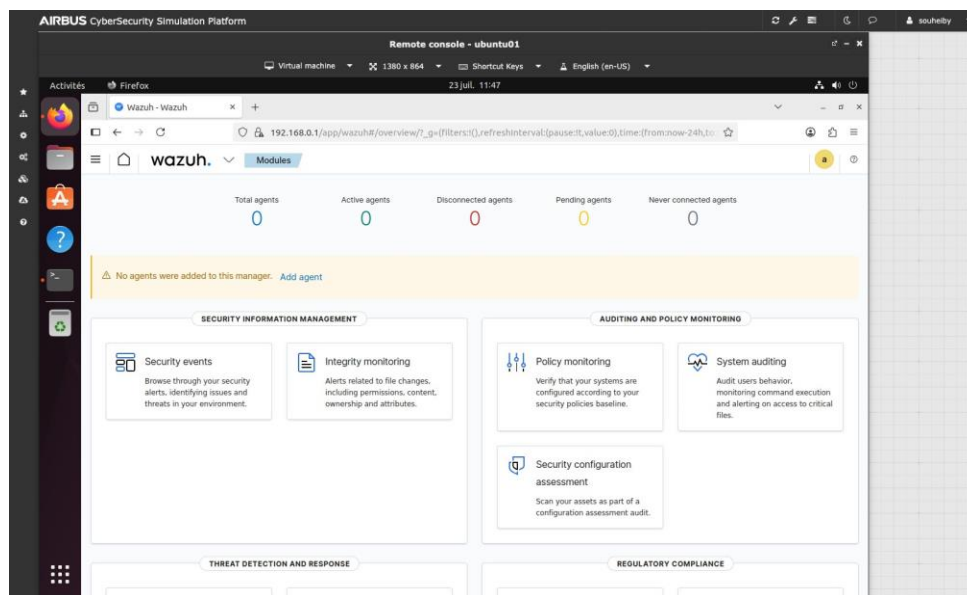
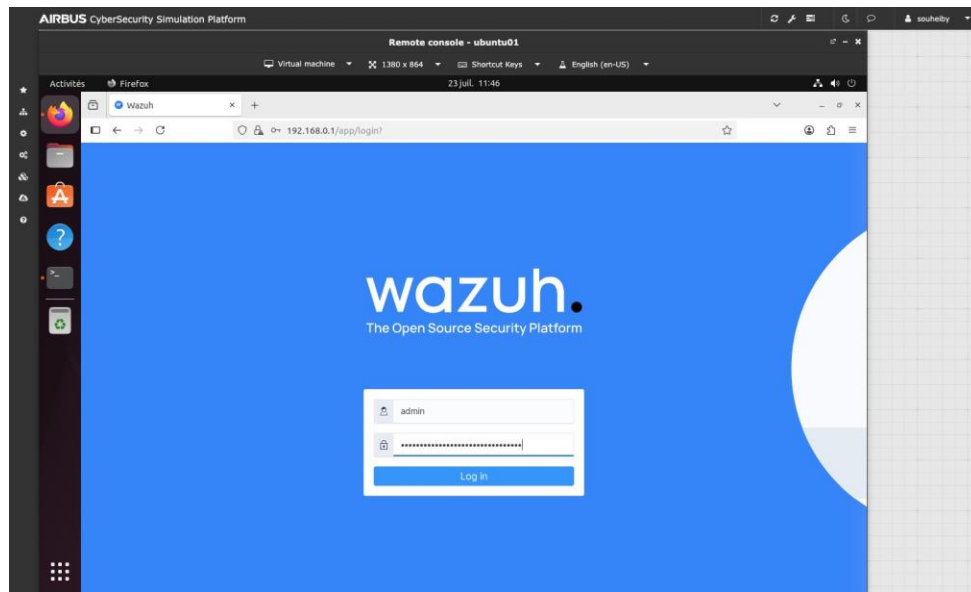
```
sudo apt install curl unzip apt-transport-https gnupg lsb-release -y
```

Ensuite, nous avons initialisé l'installation complète en mode distribué (une seule machine):



```
AIRBUS CyberSecurity Simulation Platform
Remote console - ubuntu01
Virtual machine 1380 x 864 Shortcuts Keys English (en-US)
23 Jul, 11:34
lade@ubuntu01: ~
lade@ubuntu01:~$ curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
lade@ubuntu01:~$ bash wazuh-install.sh -s
wazuh-install.sh: ligne 2585: /var/log/wazuh-install.log: Permission non accordée
^C
This script must be run as root.
lade@ubuntu01:~$ sudo bash wazuh-install.sh -a
23/07/2025 11:32:38 INFO: Starting Wazuh installation assistant. Wazuh version:
4.7.5
23/07/2025 11:32:38 INFO: Verbose logging redirected to /var/log/wazuh-install.l
og
23/07/2025 11:32:53 INFO: --- Dependencies ---
23/07/2025 11:32:53 INFO: Installing gawk.
23/07/2025 11:33:01 INFO: Wazuh web interface port will be 443.
23/07/2025 11:33:10 INFO: Wazuh repository added.
23/07/2025 11:33:10 INFO: --- Configuration files ---
23/07/2025 11:33:10 INFO: Generating configuration files.
23/07/2025 11:33:20 INFO: Created wazuh-install-files.tar. It contains the Wazuh
cluster key, certificates, and passwords necessary for installation.
23/07/2025 11:33:21 INFO: --- Wazuh indexer ---
23/07/2025 11:33:21 INFO: Starting Wazuh indexer installation.
```

L'accès au Dashboard se fait ensuite sur un navigateur par : https://IP_SERVER_PRINCIPAL



Les fichiers de configuration sont localisés comme suit :

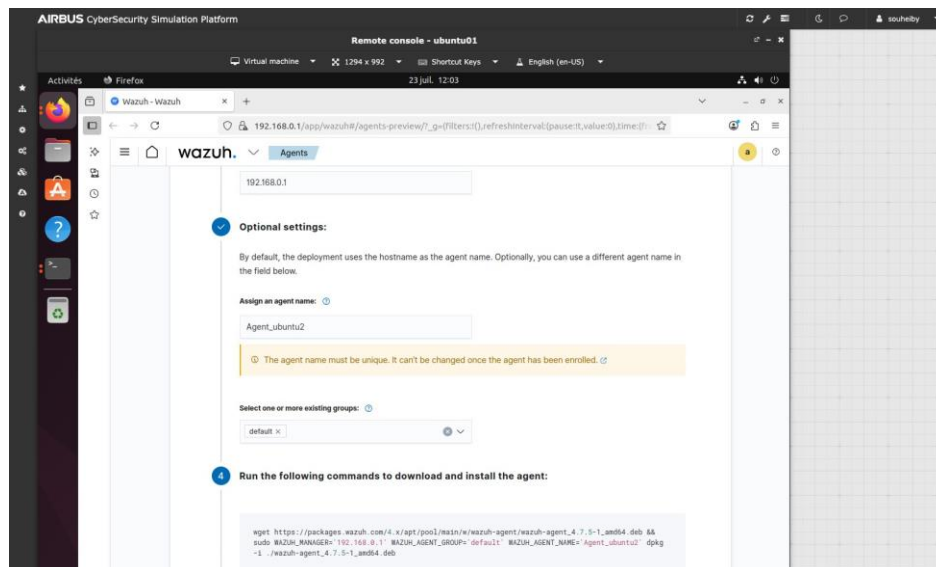
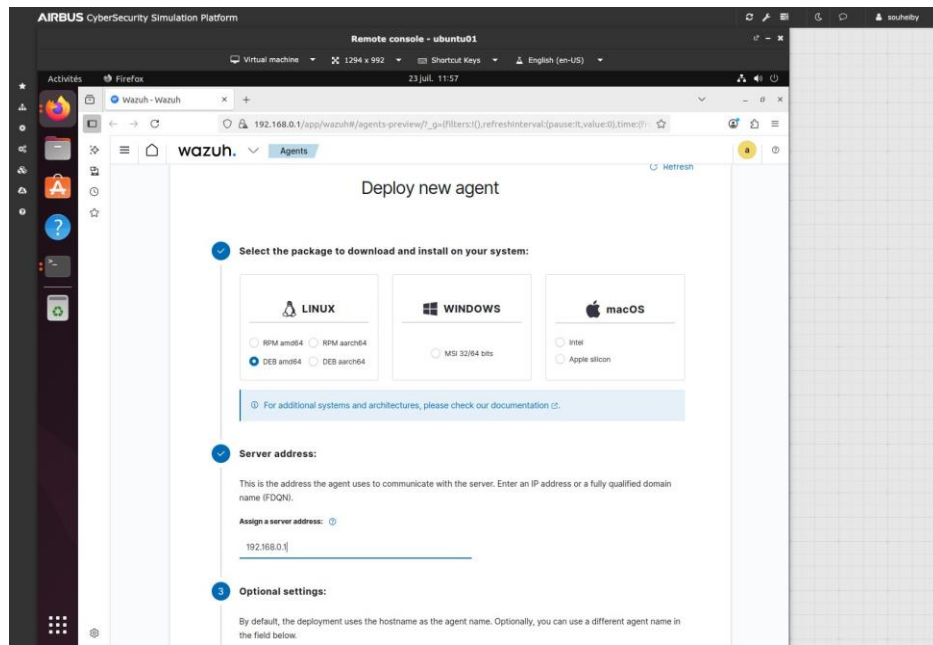
Manager : `/var/ossec/etc/ossec.conf`

Dashboard: `/etc/wazuh-dashboard/opensearch_dashboards.yml`

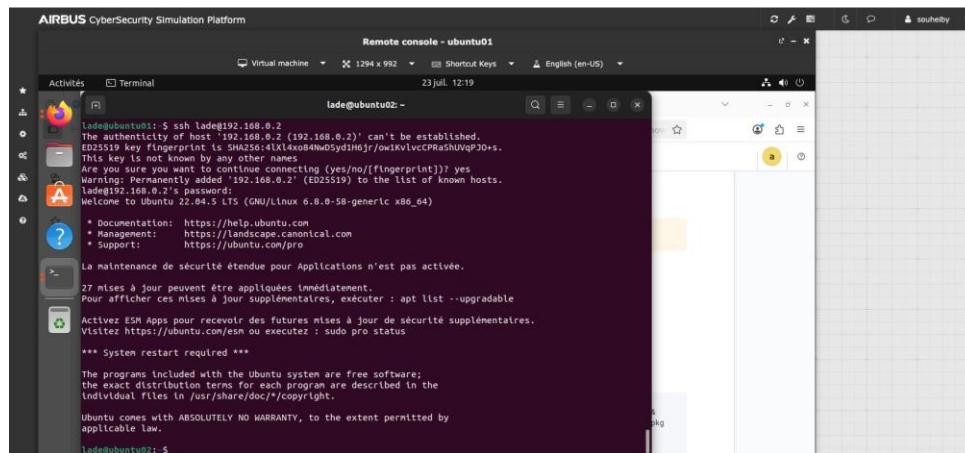
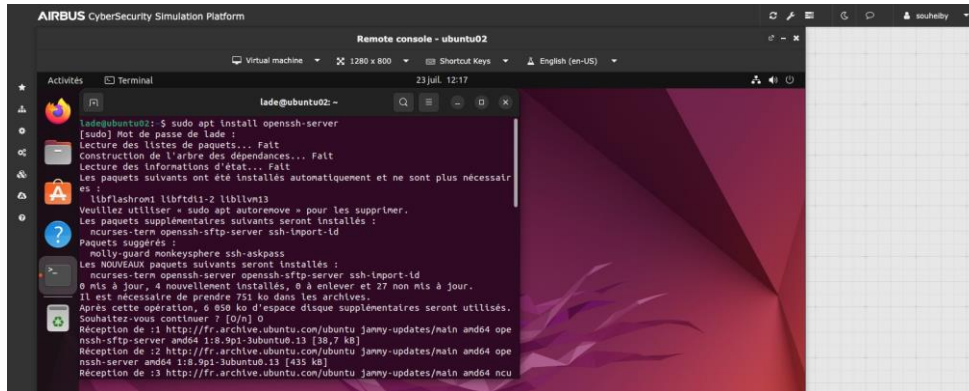
Agents Linux

Ensuite, afin de collecter les logs sur les machines et effectuer la surveillance d'intégrité, nous installons des agents sur chaque machine.

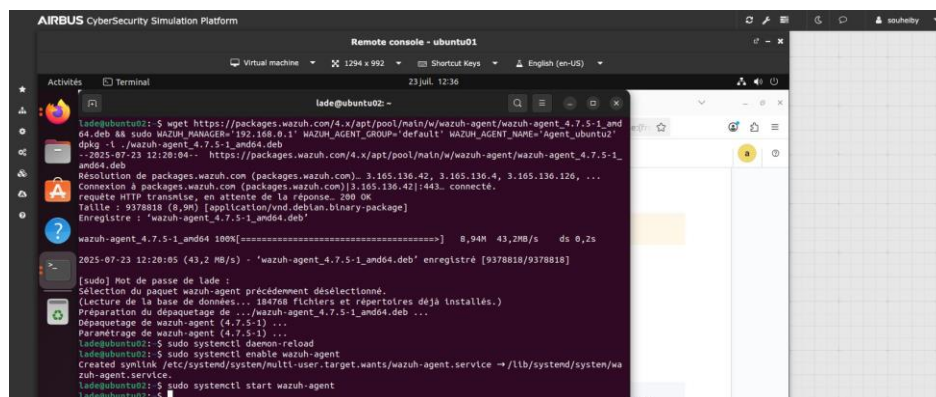
Prenons comme exemple l'agent ubuntu-2

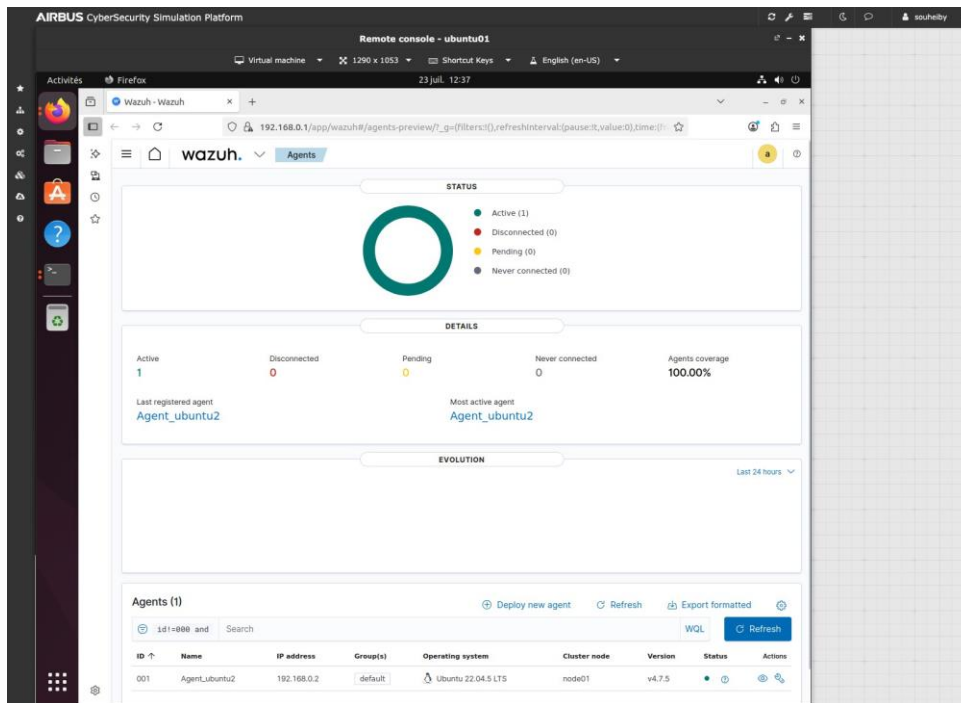


Pour remédier au problème de copier-coller entre les machines. On vous suggère d'ouvrir une session SSH entre les machines :



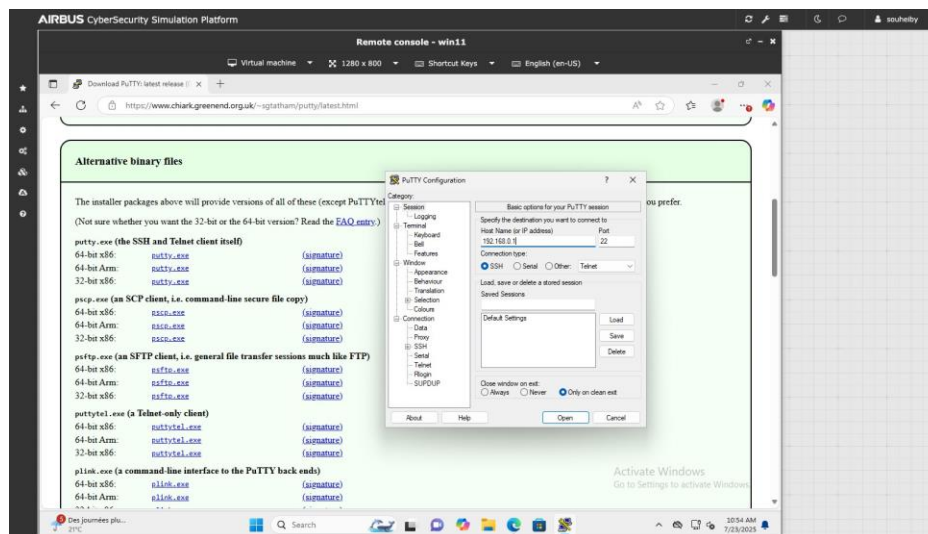
Et puis coller le script dédié pour l'installation de l'agent dans le terminal à distance de ubuntu 2 dédié pour ça. Et puis démarrer l'agent.

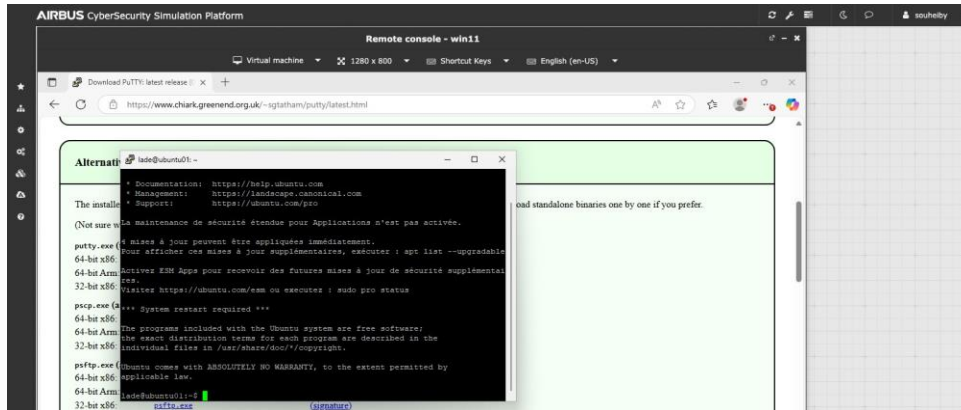




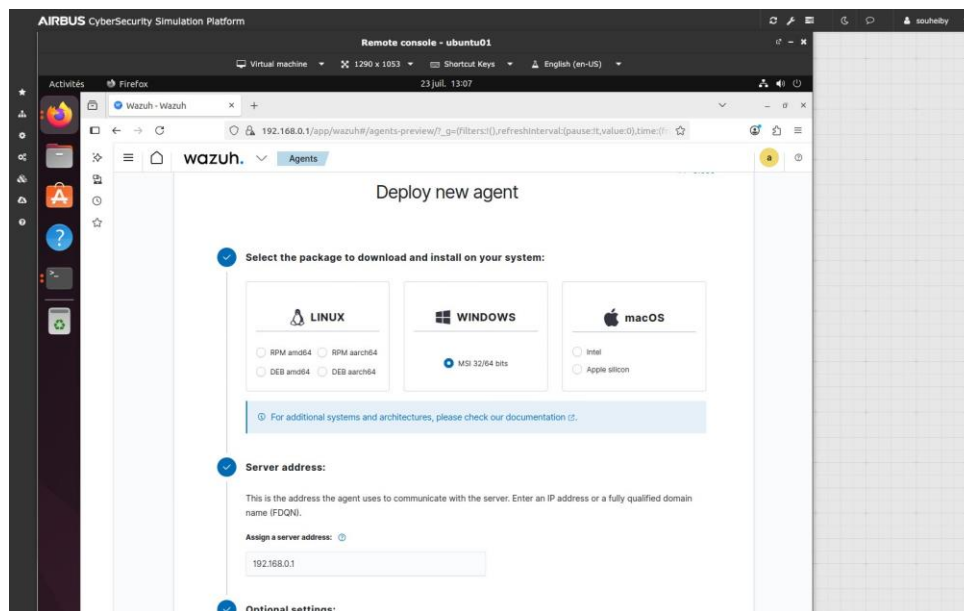
Agent Windows

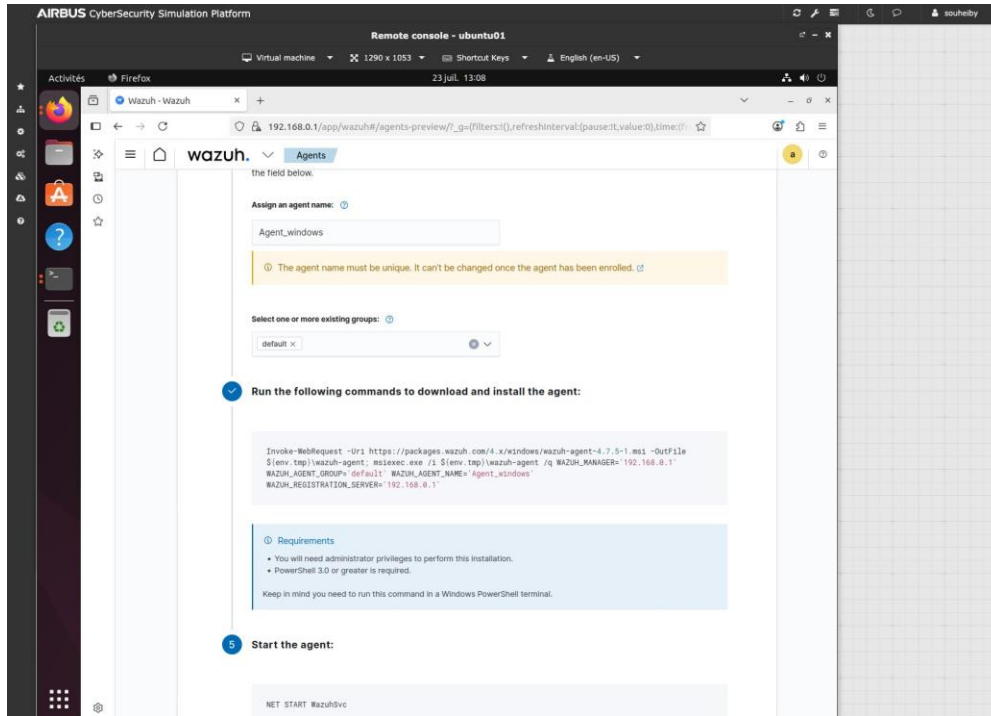
Pour avoir une représentation réaliste, nous ajoutons un agent sur un OS Windows. Cela nous démontre la scalabilité Wazuh. Pour ce faire, on installe l'outil PuTTY qui nous permettra de récupérer le script de l'agent Windows.



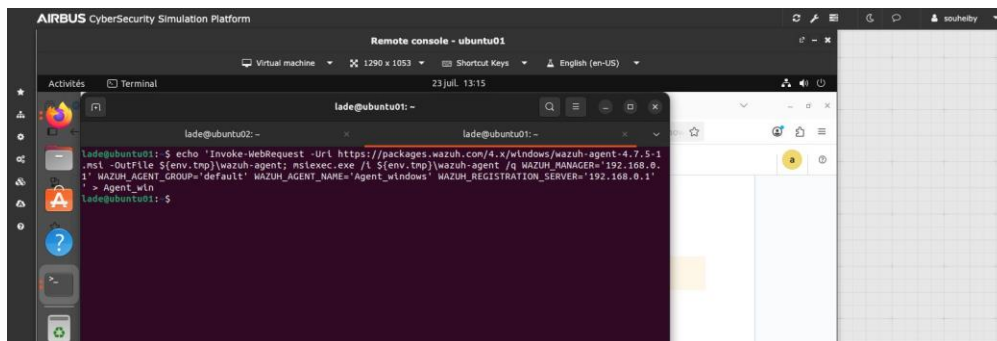


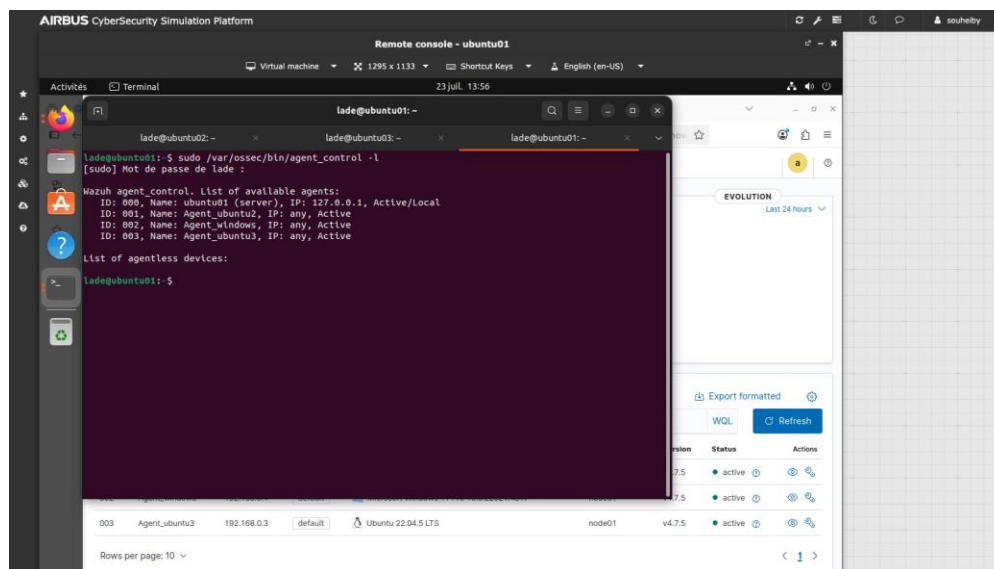
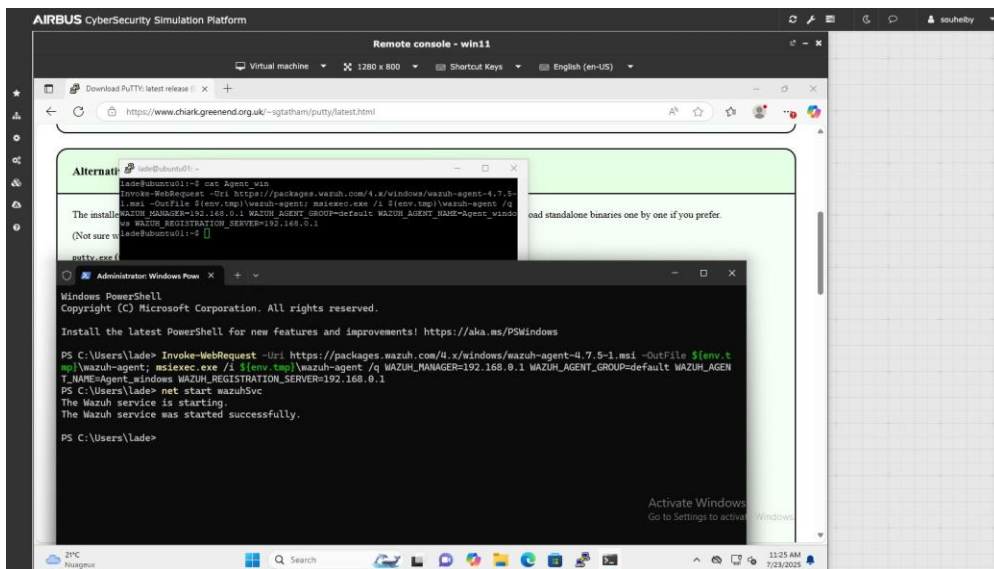
Nous lançons la création de l'agent windows à partir du manager ubuntu 1





On met le script dans un fichier Agent_win pour le récupérer à partir de la session distante Ubuntu à partir de windows.





Pour tester le fonctionnement et la détection des alertes, commençons par une attaque de brute force à partir de la machine ubuntu2.

Le SIEM Wazuh est déjà en place sur l'environnement. Le serveur tourne sur la machine ubuntu01 et les agents sont installés sur ubuntu02 et sur la machine Windows. On ne revient

pas sur cette installation. Les agents remontent bien leurs logs vers le manager et le tableau de bord est accessible.

Attaque par brute force

Le but de cette partie est de vérifier si Wazuh détecte une attaque par brute force sur le SSH. L'attaque part de la machine ubuntu02 et vise le serveur ubuntu01.

Installation de Hydra

On installe d'abord Hydra sur ubuntu02. C'est l'outil qui va essayer les mots de passe les uns après les autres sur le service SSH.

```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ sudo apt install hydra  
Lecture des listes de paquets... Fait  
Construction de l'arbre des dépendances... Fait  
Les NOUVEAUX paquets suivants seront installés :  
  hydra  
Paramétrage de hydra (9.5-1) ...  
lade@ubuntu-02:~$
```

Installation de Hydra sur ubuntu02

Creation du dictionnaire

On prépare ensuite un petit dictionnaire de mots de passe. On y met quelques mots de passe courants et le vrai mot de passe du compte visé pour que l'attaque aboutisse.

```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ cat wordlist.txt  
123456  
password  
admin  
root  
azerty  
qwerty  
toor  
letmein  
lade  
lade@ubuntu-02:~$
```

Contenu du dictionnaire utilise

Lancement de l'attaque

On lance Hydra sur le SSH de ubuntu01 avec le compte lade et le dictionnaire. L'outil teste chaque entree et finit par trouver le bon mot de passe.

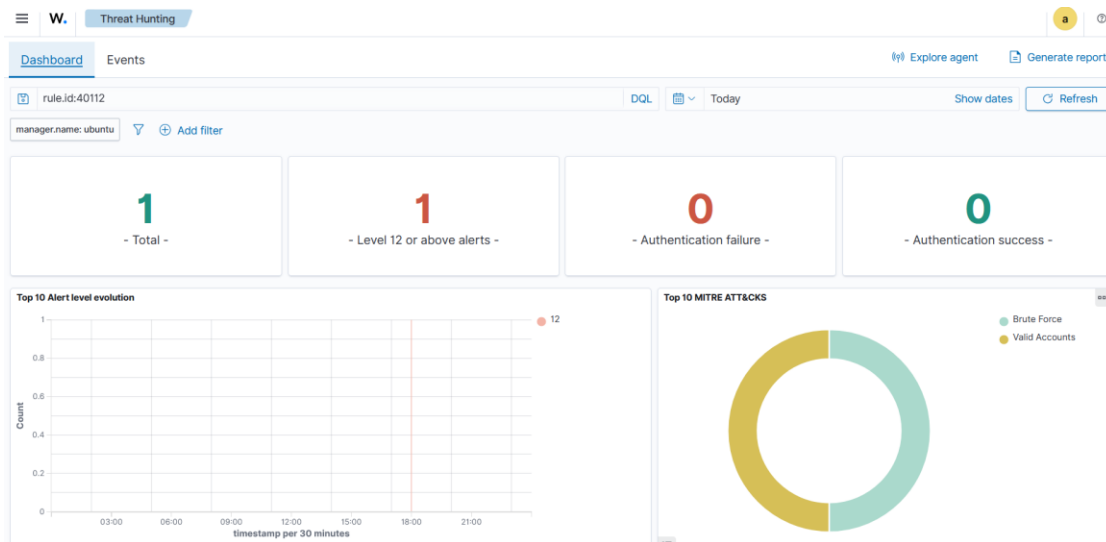
```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ hydra -l lade -P wordlist.txt ssh://192.168.0.1  
[DATA] max 9 tasks per 1 server, overall 9 tasks, 9 login tries  
[DATA] attacking ssh://192.168.0.1:22/  
[22][ssh] host: 192.168.0.1  login: lade  password: lade  
1 of 1 target successfully completed, 1 valid password found  
lade@ubuntu-02:~$
```

Resultat de l'attaque, le mot de passe est retrouve

Hydra retrouve le mot de passe du compte. Du côté du serveur, cette serie de tentatives ratees suivie d'une connexion reussie est le comportement que le SIEM doit reperer.

Detection sur Wazuh

Toutes ces tentatives sont remontees par l'agent vers le manager. Wazuh regroupe les echecs de connexion repetes et leve une alerte de brute force. On la retrouve dans le module Threat Hunting, classée dans la categorie Brute Force du referentiel MITRE.



Alerte de brute force remontee dans le tableau de bord Wazuh

Surveillance de l'integrite des fichiers

Cette partie verifie que Wazuh remonte bien les changements faits sur des fichiers surveilles. On choisit un dossier a surveiller sur l'agent Linux et sur l'agent Windows, puis on cree, on modifie et on supprime un fichier pour voir si chaque action fait reagir le SIEM.

Configuration sur l'agent Linux

On ajoute le dossier a surveiller dans le fichier de configuration de l'agent ubuntu02. L'option realtime previent tout de suite des qu'un changement a lieu et report_changes garde une trace de ce qui a ete modifie dans le fichier.

```
/var/ossec/etc/ossec.conf

<syscheck>
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>
  <directories check_all="yes" realtime="yes" report_changes="yes">
    /home/lade/fim_test</directories>
  <alert_new_files>yes</alert_new_files>
</syscheck>
```

Dossier surveille ajoute dans la configuration de l'agent Linux

On redemarre l'agent pour qu'il prenne la nouvelle configuration, puis on fait le test en creant le fichier, en lui ajoutant une ligne et en le supprimant.


```
lade@ubuntu-02: ~
lade@ubuntu-02:~$ sudo systemctl restart wazuh-agent
lade@ubuntu-02:~$ touch /home/lade/fim_test/mon_fichier.txt
lade@ubuntu-02:~$ echo "ligne ajoutée dans le fichier" >> /home/lade/fim_test/mon_fichier.txt
lade@ubuntu-02:~$ rm /home/lade/fim_test/mon_fichier.txt
lade@ubuntu-02:~$
```

Redemarrage de l'agent et test sur le fichier mon_fichier.txt

Configuration sur l'agent Windows

On fait la même chose sur la machine Windows. On ajoute un dossier à surveiller dans la configuration de l'agent, puis on redemarre le service pour l'appliquer.

```
ossec-agent\ossec.conf

<syscheck>
  <directories check_all="yes">%WINDIR%\regedit.exe</directories>
  <directories check_all="yes" realtime="yes" report_changes="yes">
    C:\fim_test</directories>
  <alert_new_files>yes</alert_new_files>
</syscheck>
```

Dossier surveillance ajouté dans la configuration de l'agent Windows

```
Windows PowerShell

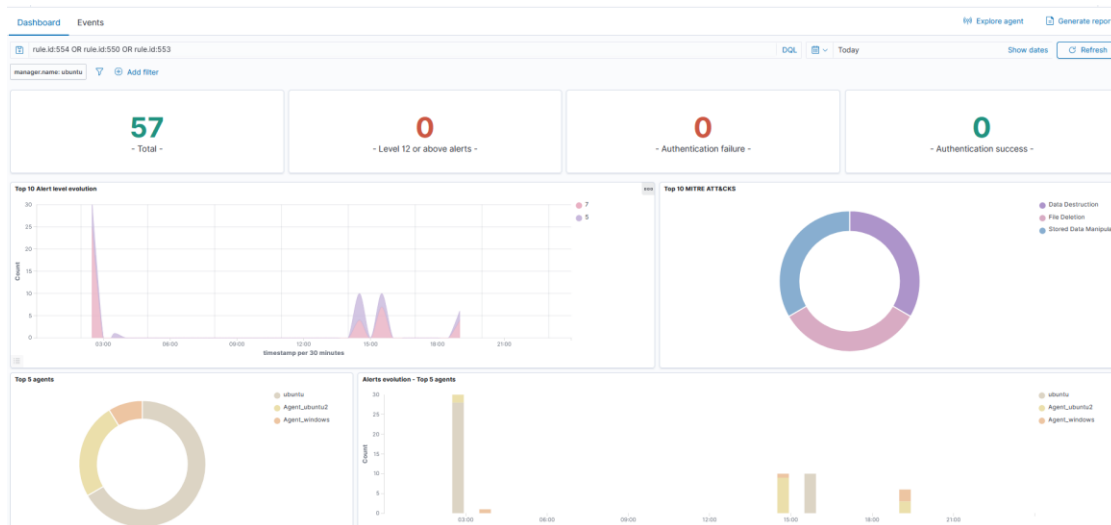
PS C:\> Restart-Service WazuhSvc
PS C:\> Get-Service WazuhSvc

Status      Name      DisplayName
-----
Running     WazuhSvc  Wazuh
PS C:\>
```

Redemarrage du service de l'agent Windows

Detection sur Wazuh

Chaque action sur le fichier remonte jusqu'au manager. Wazuh affiche trois événements séparés, un pour la création du fichier, un pour sa modification et un pour sa suppression. On les retrouve dans le tableau de bord en filtrant sur les règles 554, 550 et 553, pour les agents Linux et Windows.



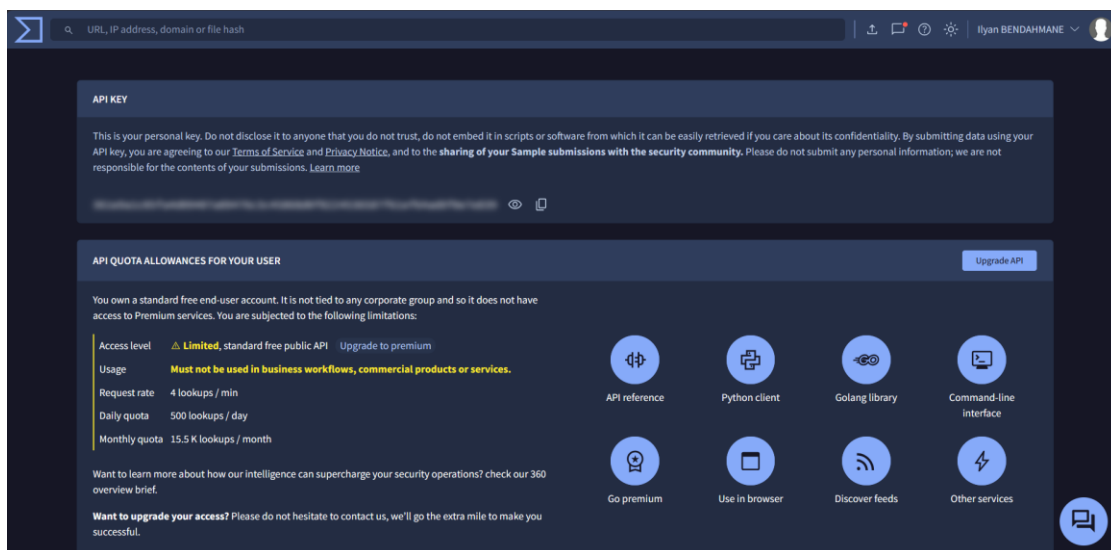
Alertes FIM remontées dans le tableau de bord pour les deux agents

Integration de VirusTotal

VirusTotal analyse un fichier a partir de son empreinte et indique s'il est connu comme malveillant. On branche Wazuh dessus pour que le manager interroge le service des qu'un nouveau fichier apparait dans un dossier surveille.

Compte et cle API

On cree un compte gratuit sur VirusTotal et on recupere la cle API depuis le profil. Cette cle sert au manager pour envoyer ses requetes.



Cle API recuperee depuis le compte VirusTotal

Ajout de l'integration sur le manager

On ajoute un bloc integration dans la configuration du manager, sur ubuntu01. On y colle la cle API et on relie l'integration au module de surveillance des fichiers pour qu'il envoie les empreintes a VirusTotal.

```
/var/ossec/etc/ossec.conf

<integration>
  <name>virustotal</name>
  <api_key>361e9a1c05fa4d09...8f0e7e839</api_key>
  <group>syscheck</group>
  <alert_format>json</alert_format>
</integration>
```

Bloc integration VirusTotal ajoute dans la configuration du manager

Dossier surveillance en temps reel

On cree un dossier software sur l'agent ubuntu02. C'est le dossier qui va servir de zone a risque pour le test.

```
lade@ubuntu-02: ~

lade@ubuntu-02:~$ mkdir /home/lade/software
lade@ubuntu-02:~$ ls -ld /home/lade/software
drwxrwxr-x 2 lade lade 4096 juin 21 20:45 /home/lade/software
lade@ubuntu-02:~$
```

Creation du dossier software sur l'agent

On ajoute ce dossier a la surveillance des fichiers en temps reel. L'option realtime previent tout de suite, report_changes garde la trace des changements, check_all verifie toutes les metadonnees et calcule les empreintes, et whodata ajoute l'identite de l'utilisateur et le programme a l'origine du changement.

```
/var/ossec/etc/ossec.conf

<syscheck>
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories check_all="yes" realtime="yes" report_changes="yes"
    whodata="yes">/home/lade/software</directories>
</syscheck>
```

Dossier software ajoute en surveillance temps reel sur l'agent

Redemarrage des services

On redemarre le manager pour prendre en compte l'integration, et l'agent pour la nouvelle zone surveillee.

```
redemarrage des services

lade@ubuntu-01:~$ sudo systemctl restart wazuh-manager
lade@ubuntu-02:~$ sudo systemctl restart wazuh-agent
```

Redemarrage du manager et de l'agent

Test avec le fichier EICAR

Pour tester sans vrai virus, on utilise le fichier EICAR. C'est un fichier inoffensif que tous les antivirus reconnaissent comme une menace de test. On le telecharge dans le dossier software sous le nom suspicious-file.exe.

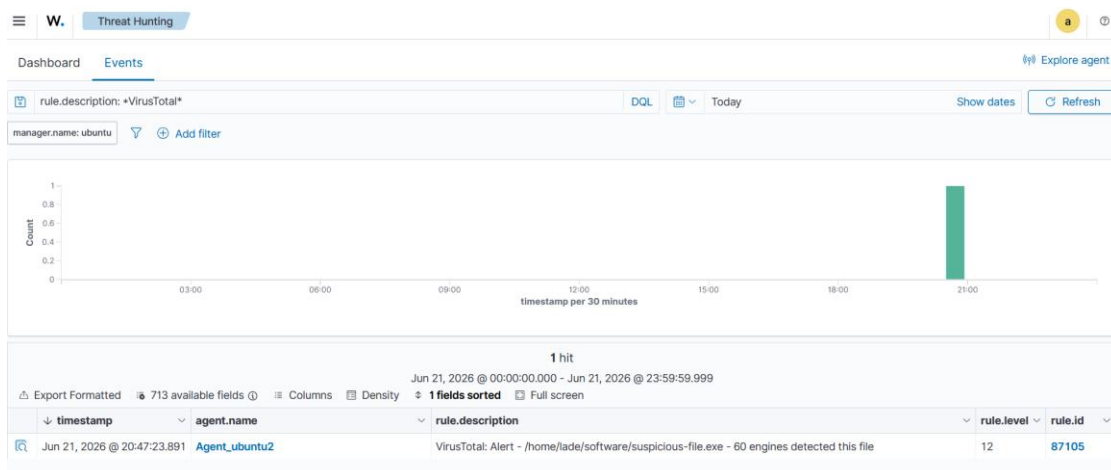
```
lade@ubuntu-02: ~

lade@ubuntu-02:~$ curl -Lo suspicious-file.exe https://secure.eicar.org/eicar.com
lade@ubuntu-02:~$ ls
suspicious-file.exe
lade@ubuntu-02:~$
```

Telechargement du fichier de test EICAR dans le dossier surveille

Detection sur Wazuh

Des que le fichier arrive dans le dossier, l'agent le signale au manager. Le manager extrait l'empreinte du fichier et l'envoie a VirusTotal. La reponse indique que le fichier est connu comme malveillant et Wazuh leve une alerte de niveau eleve.



Alerte VirusTotal remontee dans le tableau de bord, 60 moteurs detectent le fichier

Mise en place de Suricata

Suricata est un moteur d'inspection du trafic reseau. On l'installe sur ubuntu02 pour qu'il analyse les paquets et leve des alertes en cas de comportement suspect.

Installation via le depot officiel OISF

On passe par le PPA officiel de l'OISF pour avoir une version a jour de Suricata.

```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ sudo add-apt-repository ppa:oisf/suricata-stable  
lade@ubuntu-02:~$ sudo apt update  
lade@ubuntu-02:~$ sudo apt install suricata -y  
lade@ubuntu-02:~$ suricata --version  
Suricata 8.0.5  
lade@ubuntu-02:~$
```

Installation de Suricata depuis le PPA OISF sur ubuntu02

Ajout des regles Emerging Threats

Suricata a besoin de regles pour savoir quoi chercher. On telecharge le jeu de regles Emerging Threats, on le place dans le dossier de Suricata et on ajuste les permissions pour que le service puisse les lire.

```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ cd /tmp  
lade@ubuntu-02:~$ sudo curl -LO https://rules.emergingthreats.net/open/suricata-7.0.0/emerging.rules.tar.gz  
lade@ubuntu-02:~$ sudo mkdir -p /etc/suricata/rules  
lade@ubuntu-02:~$ sudo tar -xzf emerging.rules.tar.gz  
lade@ubuntu-02:~$ sudo mv rules/*.rules /etc/suricata/rules/  
lade@ubuntu-02:~$ sudo chmod 640 /etc/suricata/rules/*.rules  
lade@ubuntu-02:~$
```

Telechargement et mise en place des regles Emerging Threats

Configuration de suricata.yaml

On indique a Suricata le reseau a surveiller et les regles a charger. HOME_NET prend l'adresse de ubuntu02, EXTERNAL_NET est mis sur any, on ajoute le dossier de regles et on precise l'interface reseau active de la machine.

```
/etc/suricata/suricata.yaml

vars:
  address-groups:
    HOME_NET: "[192.168.0.2]"
    EXTERNAL_NET: "any"

rule-files:
  - suricata.rules
  - /etc/suricata/rules/*.rules

af-packet:
  - interface: enp8s0
```

Parametres principaux du fichier suricata.yaml

Redemarrage et verification

On redemarre le service et on verifie qu'il tourne bien et que les regles sont chargees.

```
lade@ubuntu-02: ~

lade@ubuntu-02:~$ sudo systemctl restart suricata
lade@ubuntu-02:~$ sudo systemctl status suricata
Active: active (running)
50300 signatures processed
lade@ubuntu-02:~$
```

Suricata actif avec les regles chargees

Attaque reseau par deni de service

On simule un deni de service de type SYN flood depuis ubuntu01 vers ubuntu02. L'option flood envoie les paquets le plus vite possible et l'option rand-source change l'adresse source a chaque paquet pour rendre le blocage plus difficile.

```
lade@ubuntu-01: ~  
  
lade@ubuntu-01:~$ sudo apt install hping3 -y  
lade@ubuntu-01:~$ sudo hping3 -c 4 -d 50 -S -w 64 -p 80 --flood --rand-source 192.168.0.2  
HPING 192.168.0.2: S set, 40 headers + 50 data bytes  
hping in flood mode, no replies will be shown  
--- 192.168.0.2 hping statistic ---  
318183 packets transmitted, 0 packets received, 100% packet loss  
lade@ubuntu-01:~$
```

Lancement du SYN flood avec hping3 depuis ubuntu01

On observe les logs de Suricata pendant l'attaque. Le fichier d'alertes se remplit avec des entrees venant d'adresses sources differentes, ce qui correspond bien a l'attaque avec des sources aleatoires.

```
/var/log/suricata/fast.log  
  
lade@ubuntu-02:~$ sudo tail -f /var/log/suricata/fast.log  
ET DROP Spamhaus DROP Listed Traffic Inbound {TCP} 124.157.11.152 -> 192.168.0.2:80  
ET DROP Spamhaus DROP Listed Traffic Inbound {TCP} 42.162.43.61 -> 192.168.0.2:80  
ET DROP Spamhaus DROP Listed Traffic Inbound {TCP} 148.178.107.53 -> 192.168.0.2:80  
ET DROP Spamhaus DROP Listed Traffic Inbound {TCP} 148.148.247.188 -> 192.168.0.2:80
```

Detection du flood par Suricata avec des adresses sources aleatoires

Remontee des alertes Suricata vers Wazuh

Pour que Wazuh exploite les alertes de Suricata, on configure l'agent pour qu'il lise le fichier de logs au format JSON produit par Suricata, le fichier eve.json.

```
/var/ossec/etc/ossec.conf  
  
<ossec_config>  
  <localfile>  
    <log_format>json</log_format>  
    <location>/var/log/suricata/eve.json</location>  
  </localfile>  
</ossec_config>
```

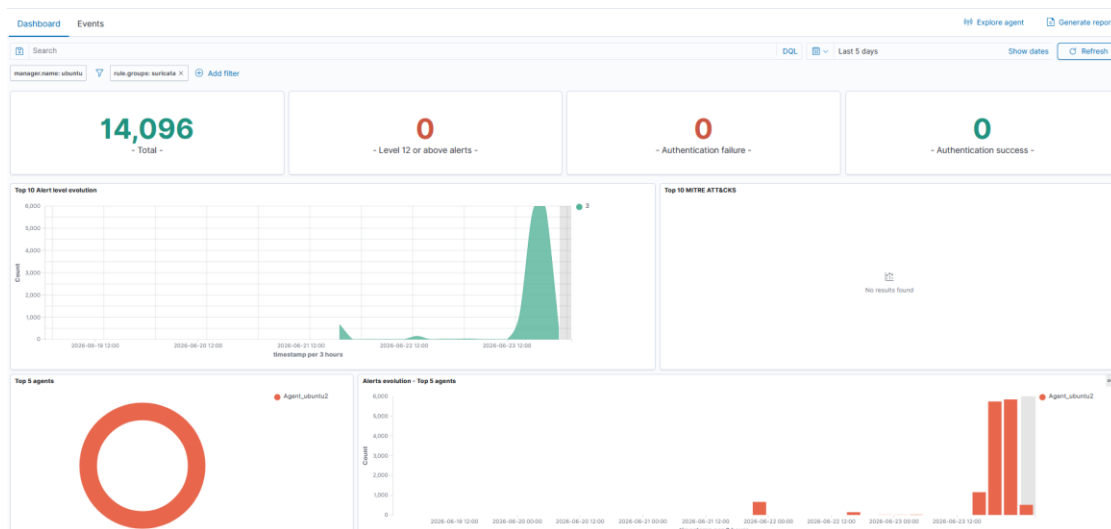
Ajout du fichier eve.json dans la configuration de l'agent

On redemarre Suricata et l'agent pour appliquer la nouvelle configuration.

```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ sudo systemctl restart suricata  
lade@ubuntu-02:~$ sudo systemctl restart wazuh-agent  
lade@ubuntu-02:~$
```

Redemarrage de Suricata et de l'agent Wazuh

Sur le tableau de bord du manager, en filtrant sur le groupe de regles suricata, on retrouve les alertes remontees par l'agent de ubuntu02.



Alertes du groupe suricata sur le tableau de bord du manager

Attaque reseau par scan de ports

On prepare un petit script qui envoie d'abord des pings puis lance plusieurs types de scans avec nmap sur ubuntu02.

script_warsh.sh

```
#!/bin/bash
cible="$1"

executer_ping() {
    ping -c 5 "$cible"
}
executer_scan_nmap() {
    nmap "$type_scan" "$cible"
}

executer_ping "$cible"
for type_scan in "-sS" "-sT" "-sV" "-sA"; do
    executer_scan_nmap "$cible" "$type_scan"
done
```

Script de scan combinant ping et nmap

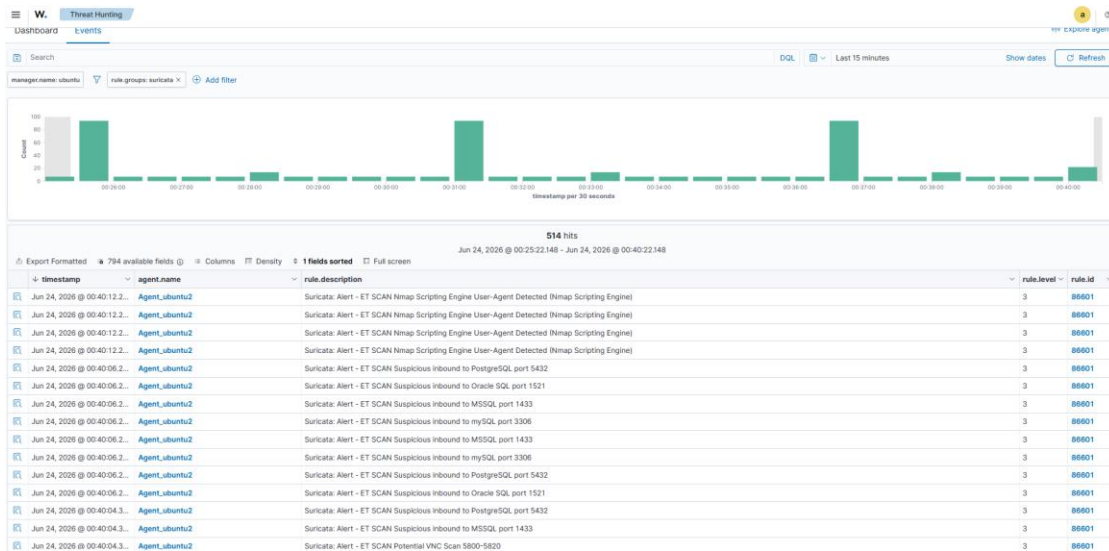
On rend le script executable, on installe nmap puis on lance le scan depuis ubuntu01 en visant ubuntu02. On voit les reponses au ping et les resultats des scans.

lade@ubuntu-01: ~

```
lade@ubuntu-01:~$ sudo chmod +x script_warsh.sh
lade@ubuntu-01:~$ sudo apt install nmap -y
lade@ubuntu-01:~$ sudo ./script_warsh.sh 192.168.0.2
Execution de ping sur 192.168.0.2 ...
5 packets transmitted, 5 received, 0% packet loss
Execution du scan Nmap avec le type de scan : -sS ...
22/tcp open  ssh
Execution du scan Nmap avec le type de scan : -sV ...
22/tcp open  ssh    OpenSSH 9.6p1 Ubuntu
Tous les scans Nmap sur 192.168.0.2 sont termines.
lade@ubuntu-01:~$
```

Execution du scan depuis ubuntu01 vers ubuntu02

Sur le tableau de bord, Wazuh remonte plusieurs alertes liées au scan. On retrouve des tentatives sur les ports 1433, 1521 et 3306, qui correspondent à des bases de données, ainsi qu'une alerte liée au ping.



Alertes de scan de ports et ICMP sur le tableau de bord

Deploiement de DVWA

Pour la partie web, on installe l'application volontairement vulnérable DVWA sur ubuntu02. Elle servira de cible pour les attaques web.

Installation de l'environnement

On installe le serveur web Apache, la base de données MariaDB et les modules PHP nécessaires.

```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ sudo apt install apache2 mariadb-server php php-mysqli php-gd libapache2-mod-php -y  
Setting up apache2 ...  
Setting up mariadb-server ...  
Setting up php8.3 ...  
lade@ubuntu-02:~$
```

Installation d'Apache, MariaDB et PHP

Base de données

On sécurise l'installation de MariaDB puis on crée une base et un utilisateur dédiés à DVWA.

```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ sudo mysql_secure_installation  
lade@ubuntu-02:~$ sudo mariadb  
MariaDB [(none)]> CREATE DATABASE dvwa;  
MariaDB [(none)]> CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'password';  
MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';  
MariaDB [(none)]> FLUSH PRIVILEGES;
```

Securisation de MariaDB et creation de la base DVWA

Recuperation et configuration de DVWA

On recupere les fichiers de DVWA depuis son depot officiel et on prepare son fichier de configuration.

```
lade@ubuntu-02: ~  
  
lade@ubuntu-02:~$ cd /var/www/html  
lade@ubuntu-02:~$ sudo git clone https://github.com/digininja/DVWA.git  
lade@ubuntu-02:~$ cd DVWA/config  
lade@ubuntu-02:~$ sudo cp config.inc.php.dist config.inc.php  
lade@ubuntu-02:~$ sudo chown -R www-data:www-data /var/www/html/DVWA  
lade@ubuntu-02:~$
```

Clonage du depot DVWA et preparation de la configuration

Dans le fichier de configuration, on renseigne l'utilisateur, le mot de passe et la base creees juste avant.

```
DVWA/config/config.inc.php  
  
$_DVWA[ 'db_server' ]    = 'localhost';  
$_DVWA[ 'db_database' ] = 'dvwa';  
$_DVWA[ 'db_user' ]     = 'dvwa';  
$_DVWA[ 'db_password' ] = 'password';
```

Parametres de connexion a la base dans config.inc.php

On active aussi une option dans le fichier php.ini, necessaire au bon fonctionnement de certaines pages de DVWA.

```
/etc/php/8.3/apache2/php.ini
```

```
; Whether to allow include/require to open URLs  
; (like http:// or ftp://) as files.  
allow_url_include = On
```

Activation de allow_url_include dans php.ini

Surveillance des logs Apache

On configure l'agent pour qu'il surveille les journaux du serveur web. C'est grâce à ces journaux que Wazuh va repérer les attaques web.

```
/var/ossec/etc/ossec.conf
```

```
<ossec_config>  
  <localfile>  
    <log_format>apache</log_format>  
    <location>/var/log/apache2/access.log</location>  
  </localfile>  
  <localfile>  
    <log_format>apache</log_format>  
    <location>/var/log/apache2/error.log</location>  
  </localfile>  
</ossec_config>
```

Ajout des logs Apache dans la configuration de l'agent

On redemarre le serveur web, la base de données et l'agent pour tout appliquer.

```
lade@ubuntu-02: ~
```

```
lade@ubuntu-02:~$ sudo systemctl restart apache2 mariadb  
lade@ubuntu-02:~$ sudo systemctl restart wazuh-agent  
lade@ubuntu-02:~$
```

Redémarrage des services et de l'agent

Initialisation et accès à DVWA

On accède à DVWA depuis un navigateur. Au premier accès, une page permet de créer la base de l'application avec le bouton Create Reset Database.



Username

Password

Login

[Damn Vulnerable Web Application \(DVWA\)](#)

Page d'initialisation de la base de DVWA

On se connecte ensuite avec le compte par défaut admin et le mot de passe password.

Non sécurisé http://100.101.128.76/DVWA/index.php

DVWA

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to practice some of the most common web vulnerabilities, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are both documented and undocumented vulnerabilities with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! Do not upload it to your hosting provider's public html folder or any Internet facing servers, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

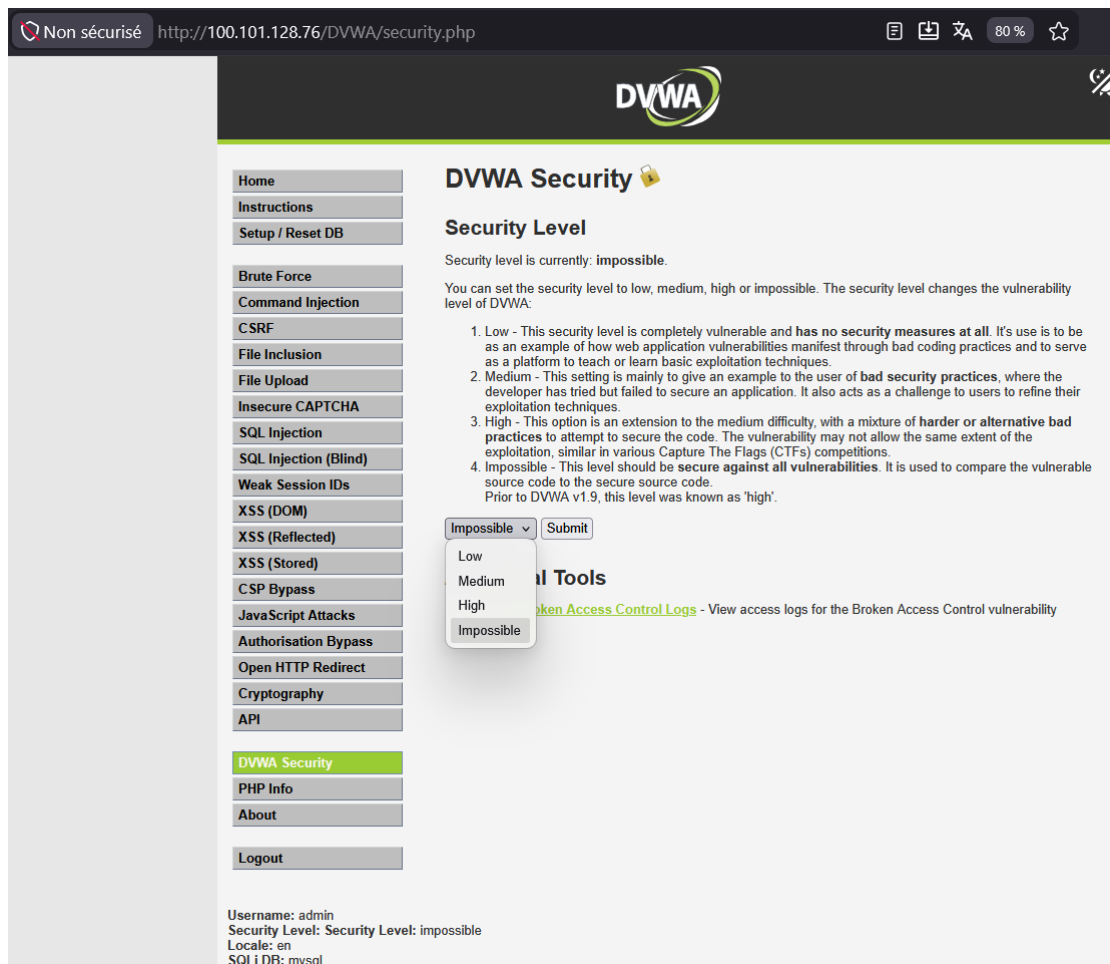
More Training Resources

DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there are plenty of other issues with web applications. Should you wish to explore any additional attack vectors, or want more difficult challenges, you may wish to look into the following other projects:

- [Mutillidae](#)
- [OWASP Vulnerable Web Applications Directory](#)

Page de connexion de DVWA

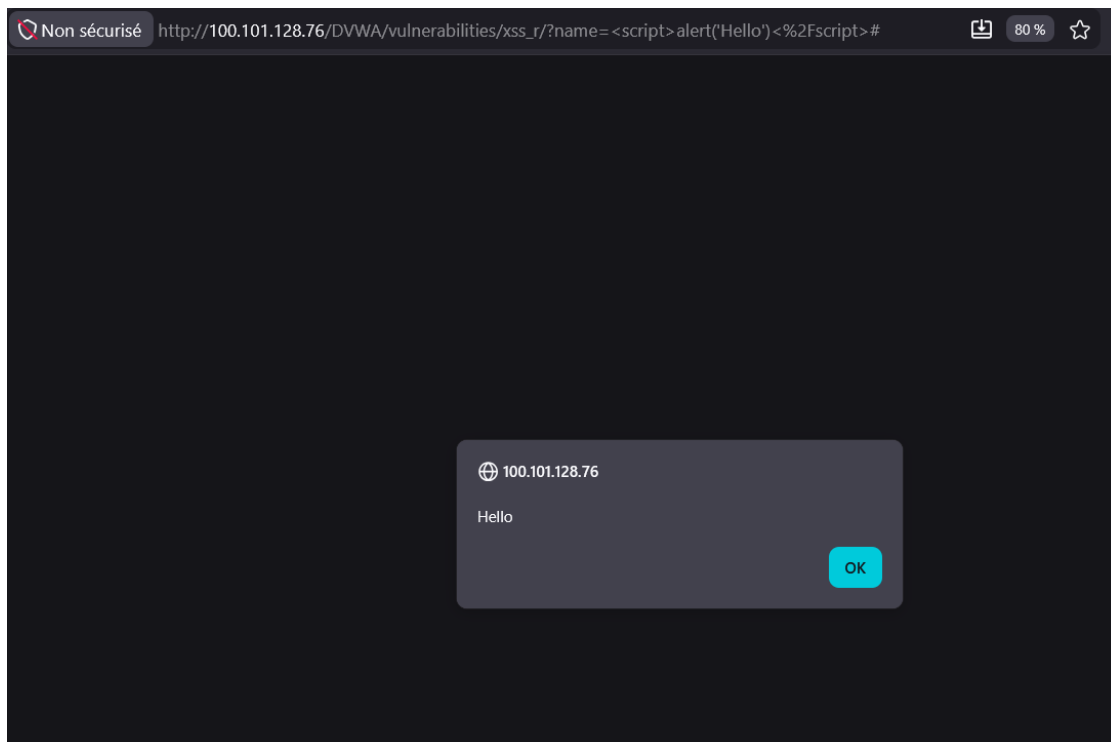
Avant de lancer les tests, on regle le niveau de securite de DVWA sur LOW pour rendre les vulnerabilites facilement exploitables.



Reglage du niveau de securite de DVWA sur LOW

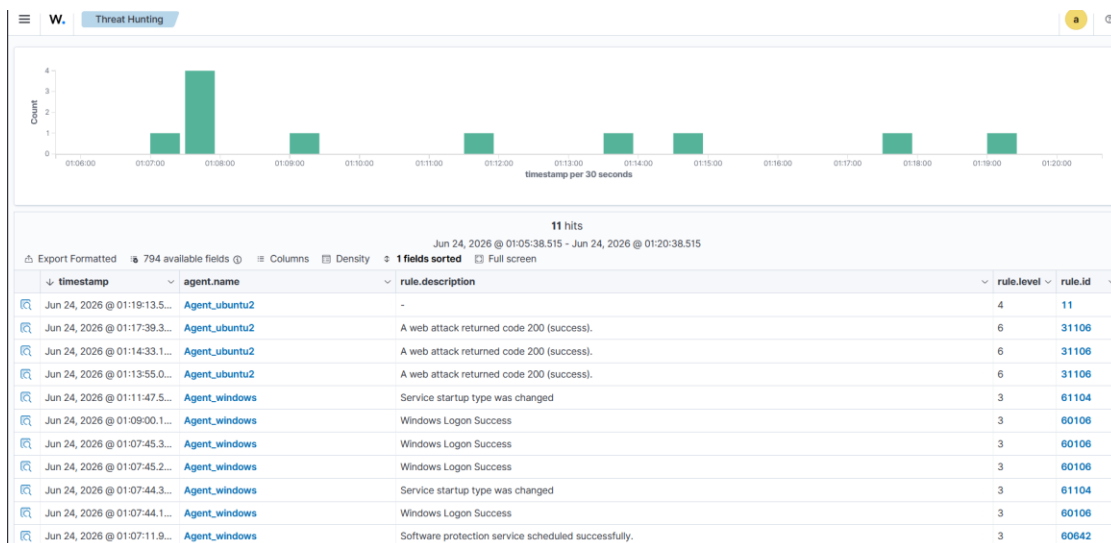
Attaque web par XSS reflechi

La premiere attaque web est un XSS reflechi. On injecte un petit script dans un champ de l'application. Avec le niveau de securite sur LOW, l'entree n'est pas filtree et le script s'execute dans le navigateur. On utilise le code `<script>alert('Hello')</script>`.



Injection XSS et fenetre Hello dans DVWA

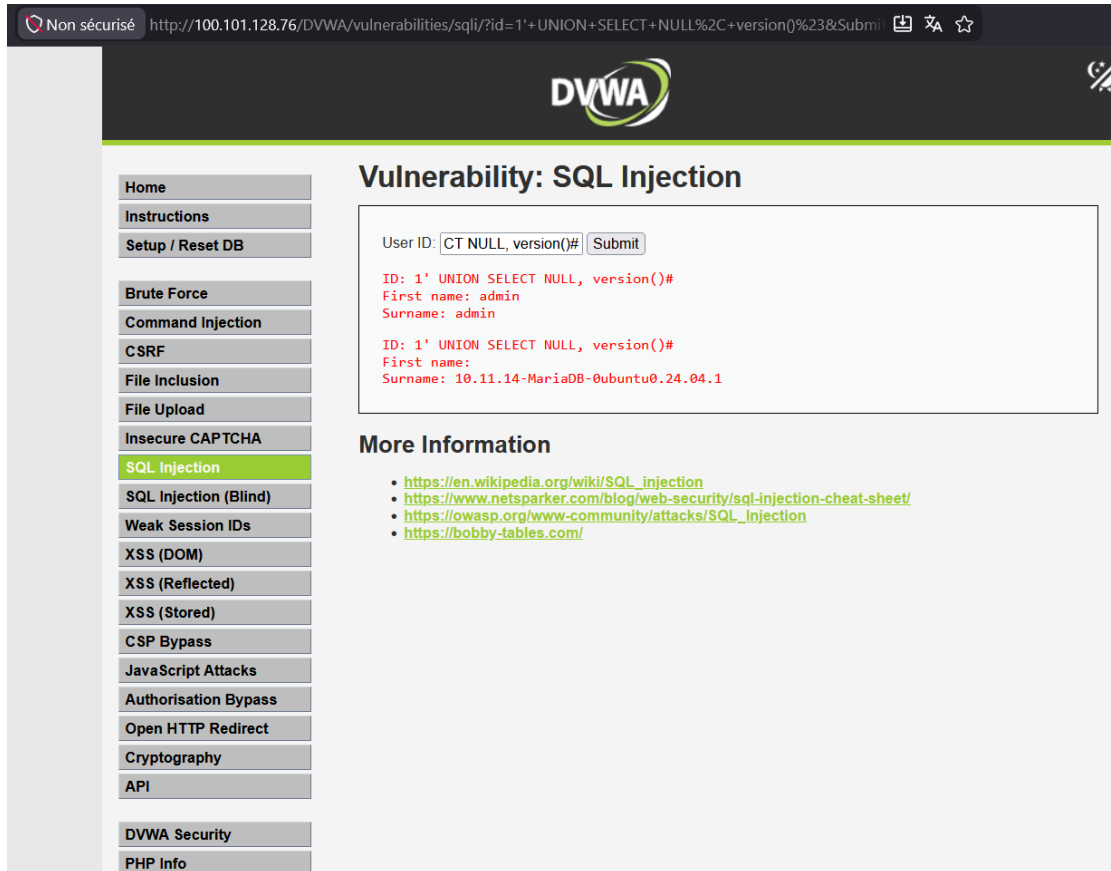
Sur le tableau de bord du manager, l'attaque est detectee et classée dans le groupe de regles web.accessing.attack, qui regroupe les tentatives d'exploitation des applications web.



Detection de l'attaque XSS sur le tableau de bord

Attaque web par injection SQL

La seconde attaque web est une injection SQL. On injecte la requete 1' UNION SELECT NULL, version()# dans un champ de saisie. La requete ferme la requete initiale et en ajoute une autre qui renvoie la version de la base, ce qui prouve que l'injection a fonctionne.



Non sécurisé http://100.101.128.76/DVWA/vulnerabilities/sqli/?id=1'+UNION+SELECT+NULL%2C+version()%23&Submit

DVWA

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT NULL, version()#
First name: admin
Surname: admin

ID: 1' UNION SELECT NULL, version()#
First name:
Surname: 10.11.14-MariaDB-0ubuntu0.24.04.1

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

DVWA Security
PHP Info

Injection SQL dans DVWA et version de la base renvoyee

La encore Wazuh detecte la tentative et la classe dans le groupe web.accessing.attack.

Enumération_web.xml

```
<group name="web,attack,">

  <rule id="100200" level="5">
    <if_sid>31101</if_sid>
    <id>^404</id>
    <description>Apache : page introuvable (404)</description>
  </rule>

  <rule id="100201" level="10" frequency="10" timeframe="20">
    <if_matched_sid>100200</if_matched_sid>
    <same_source_ip />
    <description>Enumeration de repertoires : 404 en rafale
                  depuis la meme IP</description>
  </rule>

</group>
```

Les deux regles personnalisées ajoutées au manager

La première règle, 100200, s'appuie sur la règle Wazuh 31101 qui détecte les codes d'erreur 4xx dans les journaux d'accès web. On se branche directement dessus pour ne retenir que les réponses 404. Ce point est important, car en se branchant sur la règle parente 31100 plus générale, notre règle n'était jamais évaluée et ne se déclenchait pas. Seule, elle reste discrète avec un niveau 5, car une 404 isolée n'a rien d'alarmant.

La seconde règle, 100201, est celle qui lève l'alerte. Grâce à `if_matched_sid` elle surveille les déclenchements de la règle 100200. Si dix 404 surviennent en moins de vingt secondes depuis la même adresse source, alors elle considère qu'il s'agit d'une énumération et remonte une alerte de niveau 10. Le champ `same_source_ip` garantit que le comptage se fait bien adresse par adresse.

Test de la règle

Pour déclencher la règle on prépare un petit script qui parcourt une liste de chemins fréquents et interroge le serveur web de ubuntu02 pour chacun d'eux.

```
enum.sh

#!/bin/bash
cible="http://192.168.0.2"
for chemin in admin backup config.php phpmyadmin .env \
              wp-login.php old test db secret api panel \
              uploads shell.php private; do
    code=$(curl -s -o /dev/null -w "%{http_code}" "$cible/$chemin")
    echo "GET /$chemin -> $code"
done
```

Script d'enumeration lance depuis ubuntu01

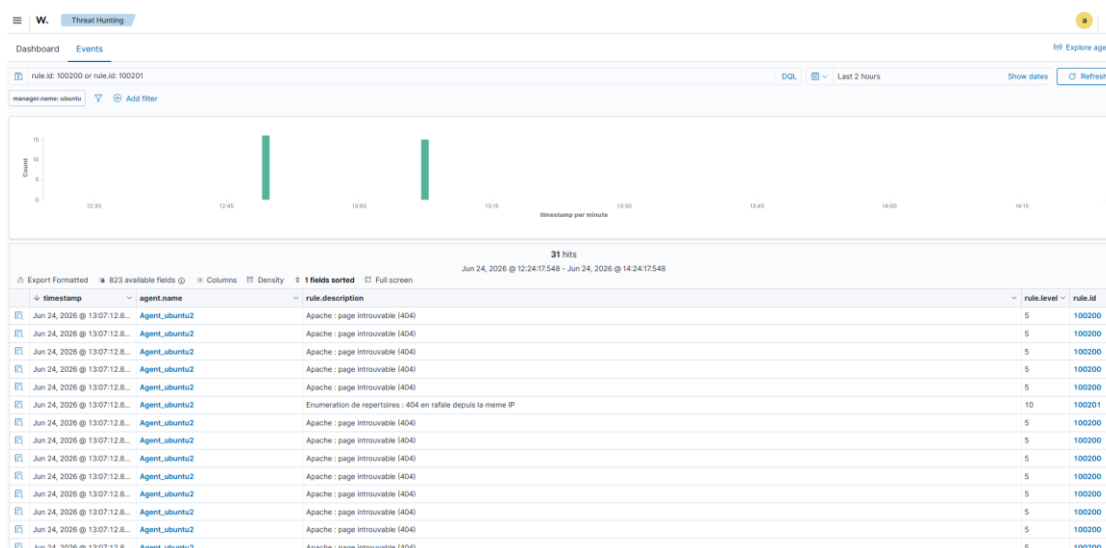
On exécute le script depuis ubuntu01 en visant ubuntu02. Comme aucun de ces chemins n'existe, le serveur renvoie une erreur 404 a chaque fois. En quelques secondes on dépasse largement le seuil de dix 404 fixe dans la règle.

```
lade@ubuntu-01: ~

lade@ubuntu-01:~$ ./enum.sh
GET /admin -> 404
GET /backup -> 404
GET /config.php -> 404
GET /phpmyadmin -> 404
GET /.env -> 404
GET /wp-login.php -> 404
GET /old -> 404
GET /test -> 404
GET /db -> 404
GET /secret -> 404
GET /api -> 404
GET /panel -> 404
GET /uploads -> 404
GET /shell.php -> 404
GET /private -> 404
```

Execution depuis ubuntu01, chaque requete renvoie un code 404

Sur le tableau de bord du manager, la règle 100201 remonte alors l'alerte d'énumération de répertoires, en indiquant l'adresse source a l'origine de la rafale.



Alerte 100201 d'enumeration et alertes 100200 sur le tableau de bord

Conclusion

Cet environnement a permis de mettre en place une chaîne de détection complète. Du côté réseau, Suricata repère le déni de service et le scan de ports, et fait remonter ses alertes à Wazuh. Du côté web, l'analyse des journaux Apache permet à Wazuh de détecter les attaques XSS et injection SQL menées sur DVWA. La corrélation des deux niveaux, réseau et application, donne une bonne vue d'ensemble des menaces dans un cadre de test contrôlé.